ML Model Deployment Strategies and Model Serving using FastAPI

Dataset: Titanic - Machine Learning from Disaster (Kaggle), train.csv

Name: _____ Roll No.: _____

1. Summary of Work

A RandomForest classifier was trained on the Titanic *train.csv* (891 rows) using an 80/20 stratified train/test split. Preprocessing (median imputation + scaling for numeric features, most-frequent imputation + one-hot encoding for categorical features) is part of a single scikit-learn Pipeline, saved once as **model.pkl** with Joblib. The same file is reused for online, batch and streaming inference and by the FastAPI service.

Item	Detail
Features	Pclass, Sex, Age, SibSp, Parch, Fare, Embarked
Model	RandomForestClassifier (200 trees, max_depth=6)
Train / Test records	712 / 179 (the 179 test records are the 'unseen' records)
Test accuracy	79.89 %
Online inference	1 record (unseen)
Batch inference	20 records, saved to batch_predictions.csv
Streaming inference	10 records, 1 record per second
FastAPI endpoints	GET /health, POST /predict (Pydantic validation)

2. Observations

How does Online Inference differ from Batch Inference?

Online inference scores a single record the moment it arrives and returns the result immediately, so it is optimised for low latency (for example, an API call from a web app). Batch inference collects many records and scores them together in one run, usually on a schedule, and optimises for throughput and efficiency rather than response time. In this project online inference predicted one passenger, while batch inference predicted 20 passengers in a single call and wrote them to batch_predictions.csv.

What is the difference between Streaming Inference and Batch Inference?

Batch inference works on a fixed, finite dataset that already exists in full, and results arrive only after the whole job finishes. Streaming inference handles a continuous flow of records, processing each one (or a tiny window) as it arrives and producing predictions with near-real-time latency. In Task 4 the 10 records were processed one at a time with a 1-second delay, whereas in Task 3 all 20 were predicted at once.

Why is FastAPI used for model serving?

FastAPI is fast (built on Starlette and async-capable), simple to write, and validates input automatically with Pydantic, which rejects bad requests with clear 422 errors before they reach the model. It also generates interactive Swagger UI documentation (/docs) for free, which makes testing easy, and it exposes the model as a language-independent REST API that any client can call.

Why should the same trained model be reused for all inference strategies?

Using one saved pipeline guarantees consistency: the same preprocessing and the same learned weights give identical predictions for a given passenger whether it is served online, in batch, in a stream or via the API. It avoids training/serving skew, makes results reproducible and comparable, saves training cost, and means there is a single artefact to version, test and monitor.

3. Conclusion

A single trained pipeline was successfully reused for three inference strategies and exposed through a validated FastAPI REST service. Valid requests returned HTTP 200 with a prediction; invalid requests returned HTTP 422 with descriptive errors.